



NUST CHIP DESIGN CENTRE

RISC-V Assembly

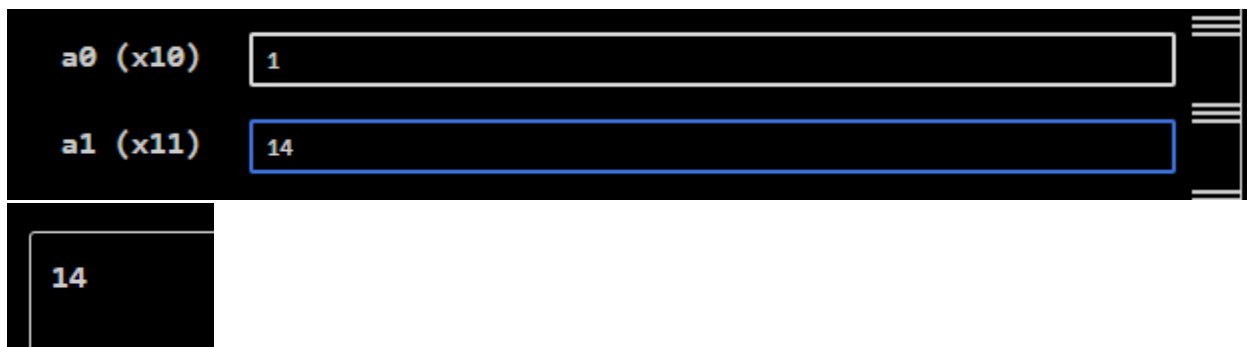
Lab Manual # 15

RISC-V Procedures and Instruction Formats (Sorting using RISC-V Assembly language)

<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>20th Aug 2025</u></i>

1. In-Lab Tasks: (Write your lab task & screenshots here)

i. Task 1:



The screenshot shows two lines of assembly code in a dark-themed editor. The first line is `a0 (x10) 1` and the second line is `a1 (x11) 14`. The value `14` in the second line is highlighted with a blue border. Below the code, there is a small window displaying the value `14`.

ii. Task 2:

<code>0x10000030</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>12</code>
<code>0x1000002C</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>2</code>
<code>0x10000028</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>11</code>
<code>0x10000024</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>6</code>
<code>0x10000020</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>7</code>
<code>0x1000001C</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>20</code>
<code>0x10000018</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>3</code>
<code>0x10000014</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>8</code>
<code>0x10000010</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>15</code>
<code>0x1000000C</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>1</code>
<code>0x10000008</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>9</code>
<code>0x10000004</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>5</code>
<code>0x10000000</code>	<code>0</code>	<code>0</code>	<code>0</code>	<code>12</code>

0x1000002C	0	0	0	20
0x10000028	0	0	0	15
0x10000024	0	0	0	12
0x10000020	0	0	0	11
0x1000001C	0	0	0	9
0x10000018	0	0	0	8
0x10000014	0	0	0	7
0x10000010	0	0	0	6
0x1000000C	0	0	0	5
0x10000008	0	0	0	3
0x10000004	0	0	0	2
0x10000000	0	0	0	1

Original Array:

12 5 9 1 15 8 3 20 7 6 11 2

After Bubble Sort:

1 2 3 5 6 7 8 9 11 12 15 20

iii. Task 3:

0x10000024	0	0	0	78
0x10000020	0	0	0	1
0x1000001C	0	0	0	2
0x10000018	0	0	0	11
0x10000014	0	0	0	4
0x10000010	0	0	0	5
0x1000000C	0	0	0	35
0x10000008	0	0	0	8
0x10000004	0	0	0	22
0x10000000	-1	-1	-1	-1

0x10000024	0	0	0	78
0x10000020	0	0	0	35
0x1000001C	0	0	0	22
0x10000018	0	0	0	11
0x10000014	0	0	0	8
0x10000010	0	0	0	5
0x1000000C	0	0	0	4
0x10000008	0	0	0	2
0x10000004	0	0	0	1
0x10000000	-1	-1	-1	-1

```
Original Array:
```

```
-1 22 8 35 5 4 11 2 1 78
```

```
After Quick Sort:
```

```
-1 1 2 4 5 8 11 22 35 78
```

2. Critical Analysis: (Write you critical analysis / conclusion here)

In this lab, we explored three different algorithms implemented in RISC-V assembly. First, the GCD demonstrated how iterative arithmetic operations can efficiently solve mathematical problems. Next, the Bubble Sort algorithm was implemented to sort an array using repeated adjacent comparisons and swaps, which reinforced the use of loops and conditional branches but had higher time complexity.

Finally, the Quick Sort algorithm provided a more efficient divide-and-conquer approach, utilizing partitioning and recursion concepts for faster sorting compared to Bubble Sort. Overall, these tasks strengthened our understanding of control flow, memory access, and algorithmic efficiency in assembly programming.